Pontificia Universidad Javeriana · Administración de Bases de Datos · 2026-30

Entrega 1: Diseño del esquema NoSQL y plan de implementación

Potencial solar en techos de los municipios PDET (UPME)

Derek Sarmiento · José Medina · Miguel Duarte · Sebastián Sánchez Vásquez · Samuel Emperador

Prof. Andrés Oswaldo Calderón Romero, Ph.D. · Lunes 2 de noviembre de 2026

Repositorio: `https://github.com/dba-upme-pdet/techos-solares-pdet`

1. Objetivo y alcance

La UPME necesita ordenar los 170 municipios PDET según el área de techos disponible para paneles solares. Para cada municipio vamos a calcular cuántas edificaciones tiene y cuánta área de techo suman, con dos fuentes abiertas de huellas de edificaciones (Microsoft Building Footprints y Google Open Buildings v3) y MongoDB como motor NoSQL. El ranking final ordena los municipios por `total_area_m2` y se presenta por separado para cada fuente.

Esta entrega define la metodología y el modelo de datos sobre los que se construyen las entregas 2 a 5: (a) plan de implementación, (b) diagrama E-R y (c) diccionario de datos. Todavía no se cargan datos; el esquema ya se puede crear con `scripts/00_init_db.py` (sección 6).

2. Decisiones tecnológicas

2.1. Motor NoSQL: MongoDB 7

MongoDB guarda geometrías como GeoJSON, las indexa con `2dsphere` y resuelve punto en polígono con `$geoWithin`. El conteo y la suma por municipio se hacen en su pipeline de agregación, sin sacar los datos de la base. Corre en Docker (`docker-compose.yml`).

Tabla 1: Motores NoSQL evaluados.

Motor	Tipos geográficos	Índice	Agregación por polígono	Decisión
MongoDB	Point, Polygon, MultiPolygon (GeoJSON)	2dsphere	`$geoWithin` + `$group`	Elegido
Elasticsearch	geo_point, geo_shape	Árbol BKD	geo_shape + agregaciones	Respaldo; exige más memoria y configuración
Cassandra	Ninguno (requiere GeoMesa)	Externo	Débil	Descartado
Neo4j	Solo puntos	De puntos	No admite polígonos	Descartado

MongoDB no calcula áreas geodésicas. El área de cada huella se calcula en Python (`pyproj.Geod`) durante la carga y se guarda como campo.

2.2. Fuentes de datos

Tabla 2: Fuentes seleccionadas. Las dos de edificaciones entregan polígonos 2D para toda Colombia, así que se cuentan y miden con el mismo método.

Fuente	Formato	Atributos clave	Rol
Microsoft Building Footprints (ODbL)	GeoJSON Lines por quadkey; índice `dataset-links.csv`	polígono, `confidence`, `height` (−1 si no se conoce)	Fuente A
Google Open Buildings v3 (CC BY-4.0 / ODbL)	CSV.gz por celda S2	polígono WKT, `area_in_meters`, `confidence`, `full_plus_code`	Fuente B
GlobalBuildingAtlas (TUM)	Teselas 3D LoD1	polígono + altura (3D); el proyecto solo necesita la huella 2D	No se usa
DANE MGN 2025, nivel municipal	Shapefile en ZIP (1,5 GB), EPSG:4686	`MPIO_CDPMP`, `MPIO_CNMBR`, `MPIO_NAREA`	Límites
`MunicipiosPDET.xlsx` (curso)	Excel, 170 filas	subregión, código DANE	Filtro PDET

El MGN y la lista PDET se cruzan por el código DIVIPOLA de 5 dígitos, nunca por nombre. El campo `area_in_meters` de Google sirve para validar el área que calculamos.

3. Plan de implementación

Cada paso es un script numerado en `scripts/` que lee rutas, URLs y umbrales de `config.yaml`, para correr el flujo de principio a fin sin pasos manuales.

Tabla 3: Fases del flujo, una por entrega.

Fase (entrega)	Pasos	Scripts y salida
0. Entorno (E1)	MongoDB 7 en Docker; Python 3.11 con `pymongo`, `geopandas`, `shapely`, `pyproj`; crear colecciones, validadores e índices. Verificar la lista PDET: 170 municipios, 16 subregiones.	`00_check_environment.py` `00_check_pdet_list.py` `00_init_db.py`
1. Límites PDET (E2)	Descargar el MGN 2025 y extraer la capa municipal. Rellenar los códigos del Excel a 5 dígitos (5031 → 05031), cruzar y registrar los que no crucen. Reproyectar de EPSG:4686 a EPSG:4326, reparar geometrías con `make_valid`, revisar el límite de 16 MB por documento.	`01_download_mgn.py` `02_load_municipalities.py` → `municipalities`
2. Huellas (E3)	Descargar solo las particiones (quadkey, celda S2) que tocan los rectángulos envolventes PDET. Leer por bloques y descartar lo que cae fuera. Por edificación: validar polígono, calcular área geodésica y centroide. `insert_many` no ordenado en lotes de 10 000; índices después de la carga. EDA.	`03_download_buildings.py` `04_load_buildings.py` `05_eda.ipynb` → `buildings`, `ingestion_runs`
3. Conteo y área (E4)	Asignar cada edificación al municipio que contiene su centroide (`$geoWithin`); aplicar confianza y área mínimas; `$group` por municipio y fuente. Exportar tablas CSV y mapas coropléticos.	`06_rooftop_stats.py` `07_maps.ipynb` → `rooftop_stats`, `outputs/`
4. Informe (E5)	Metodología, resultados, comparación de fuentes, ranking de municipios y recomendaciones para la UPME.	`docs/delivery5/`

Bordes municipales. Asignar por centroide pone cada edificación en un solo municipio, así que una edificación sobre un límite no se cuenta dos veces. La fase 3 solo llena `mpio_code` si sigue vacío, de modo que un centroide justo sobre un borde compartido queda en el primer municipio procesado. Si el centroide no cae en ningún municipio PDET, `mpio_code` queda en `null` y la edificación no entra en el conteo.

Umbrales pendientes. Área mínima (candidato: 20 m^2) y confianza mínima (candidato: 0,70 para Google; el campo de Microsoft suele venir vacío). Se fijan con el EDA de la Entrega 3. El área de huella es un límite superior del área útil para paneles: no descuenta orientación, sombra ni pendiente. El informe final lo discute.

4. Modelo de datos

Seis entidades, cada una guardada como una colección. `BUILDING` es la entidad central y la de mayor volumen (millones de documentos).

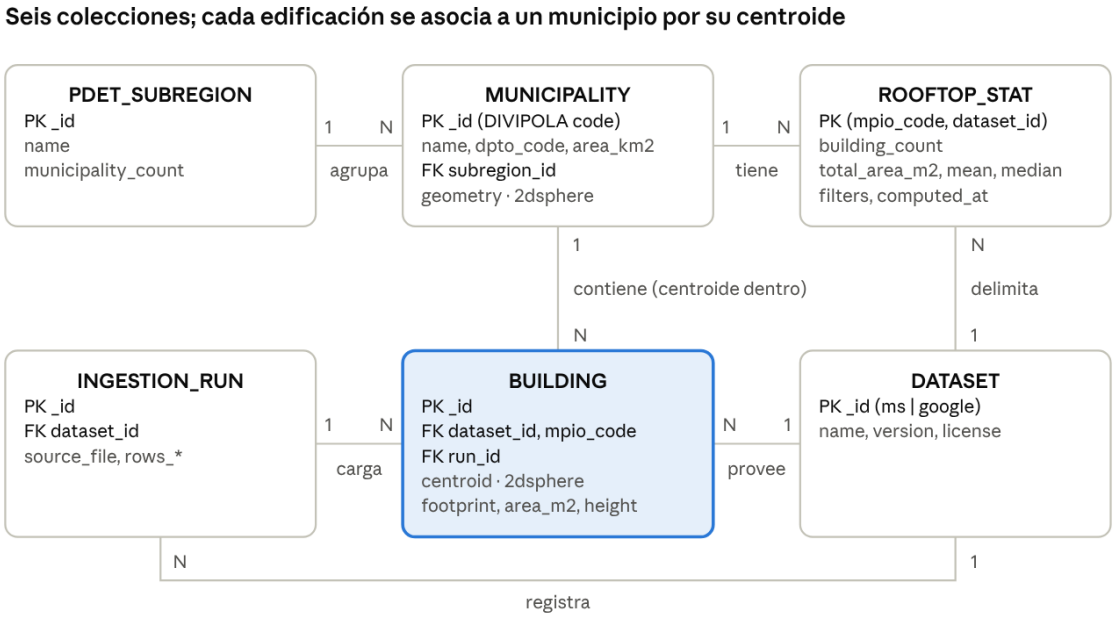


Figura 1: Diagrama E-R: 6 entidades y 7 relaciones uno a muchos. La relación MUNICIPALITY–BUILDING es espacial: `mpio_code` se llena con `$geoWithin` en la fase 3, no durante la carga. Fuente editable: `docs/delivery1/diagrama_er.md`.

Tabla 4: Del modelo E-R a documentos de MongoDB. Las 7 relaciones se agrupan en 5 patrones.

Relación	Patrón	Motivo
SUBREGION → MUNICIPALITY	Referencia `subregion_id` + copia de `subregion_name`	Los reportes agrupan por subregión sin `$lookup`
MUNICIPALITY → BUILDING	Referencia `mpio_code` en cada edificación	Millones de edificaciones no caben embebidas (16 MB por documento)
DATASET → BUILDING	Referencia `dataset_id`	Las dos fuentes en una colección: la misma consulta las compara
MUNICIPALITY + DATASET → ROOFTOP_STAT	`_id` compuesto `{mpio_code, dataset_id}`	Un resultado por municipio y fuente; recalcular sobrescribe
DATASET → INGESTION_RUN → BUILDING	Referencias `dataset_id`, `run_id`	Cada edificación se rastrea hasta el archivo y la carga que la insertó

5. Diccionario de datos

Los nombres de colecciones y campos quedan en inglés, igual que en el código. * indica campo obligatorio, exigido por el validador `$jsonSchema`.

`pdet_subregions`

Campo	Tipo	Descripción
`_id`*	int (1–16)	Columna `id` de `MunicipiosPDET.xlsx`
`name`*	string	Nombre de la subregión, p. ej. "Alto Patía y Norte del Cauca"
`municipality_count`*	int	Municipios PDET de la subregión

`municipalities`

Campo	Tipo	Descripción
`_id`*	string (5 dígitos)	Código DIVIPOLA (`MPIO_CDPMP`), p. ej. `19050`
`name`*	string	Nombre (`MPIO_CNMBR`)
`dpto_code`*, `dpto_name`*	string (2 dígitos), string	Departamento (`DPTO_CCDGO`)
`subregion_id`*, `subregion_name`*	int, string	Referencia a `pdet_subregions` y copia del nombre
`area_km2`*	double	Área oficial (`MPIO_NAREA`)
`geometry`*	GeoJSON Polygon / MultiPolygon	Límite en WGS84; índice `2dsphere`
`source`*, `loaded_at`*	object, date	`{dataset: "MGN", version: "2025", file}` y fecha de carga

`datasets`

Campo	Tipo	Descripción
`_id`*	string	`ms`, `google` o `mgn` (límites DANE)
`name`*, `version`*, `license`*	string	Nombre, versión (p. ej. `v3`) y licencia
`url`*, `accessed_at`*	string, date	Fuente de descarga y fecha de acceso

`buildings`

Campo	Tipo	Descripción
`_id`*	ObjectId	Generado por MongoDB
`dataset_id`*	string	Referencia a `datasets`
`source_ref`	string	ID original (Google: `full_plus_code`; Microsoft: archivo + fila)
`footprint`*	GeoJSON Polygon	Contorno del techo en WGS84
`centroid`*	GeoJSON Point	Centroide; índice `2dsphere`; base de la asignación
`area_m2`*	double	Área geodésica (`pyproj.Geod`)
`area_src_m2`	double	Área reportada por Google, para validar
`confidence`	double (0–1)	Confianza de la detección; `null` si no viene
`height_m`	double	Altura de Microsoft; `null` si es −1
`mpio_code`	string (5 dígitos)	Referencia a `municipalities`; se llena en la fase 3
`run_id`*	ObjectId	Referencia a `ingestion_runs`

rooftop_stats

Campo	Tipo	Descripción
`_id`*	object	`{mpio_code, dataset_id}`
`building_count`*	int	Edificaciones con centroide en el municipio, tras filtros
`total_area_m2`*, `mean_area_m2`*, `median_area_m2`*	double	Suma, promedio y mediana de `area_m2`
`filters`*, `computed_at`*	object, date	`{min_area_m2, min_confidence}` aplicados y fecha del cálculo

ingestion_runs

Campo	Tipo	Descripción
`_id`*	ObjectId	Generado por MongoDB
`dataset_id`*, `source_file`*	string	Referencia a `datasets` y archivo o partición cargada
`rows_read`*, `rows_loaded`*, `rows_rejected`*	int	Contadores de la carga
`started_at`*, `finished_at`*	date	Inicio y fin de la carga
`notes`	string	Errores o advertencias

6. Índices y validación del esquema

Tabla 5: Índices. Indexar el centroide (un punto) y no el polígono hace el índice más pequeño y la prueba de punto en polígono más barata sobre millones de documentos.

Colección	Clave	Tipo	Uso
`municipalities`	`geometry`	2dsphere	Consultas espaciales contra los límites
`municipalities`	`subregion_id`	B-tree	Agrupar por subregión
`buildings`	`centroid`	2dsphere	Asignación con `$geoWithin` (consulta principal)
`buildings`	`{dataset_id, mpio_code}`	Compuesto	Agregar por fuente y municipio
`ingestion_runs`	`dataset_id`	B-tree	Auditar cargas por fuente

`scripts/00_init_db.py` crea las seis colecciones con validadores `$jsonSchema` (campos obligatorios y tipos de la sección anterior) y los índices de la tabla. Los índices de `buildings` se crean con `--building-indexes` después de la carga masiva, para no frenar las inserciones. Lo probamos contra MongoDB 7.0:

- Se crean las 6 colecciones y los 5 índices (los 2 de `buildings` con `--building-indexes`); correrlo dos veces no cambia nada.
- Un municipio con código `5031` (4 dígitos) es rechazado por el validador.
- `$geoWithin` sobre `buildings.centroid` usa el índice `centroid_2dsphere` según `explain()`.

7. Riesgos y mitigaciones

Riesgo	Mitigación
Los archivos de edificaciones cubren todo el país y no caben en memoria	Descargar solo particiones que tocan los rectángulos PDET; leer por bloques; filtrar antes de insertar
Polígonos inválidos: el índice `2dsphere` de `municipalities` los rechaza, y una huella inválida da área y centroide errados	`make_valid` antes de insertar; `rows_rejected` y `notes` en `ingestion_runs`
Los nombres de la lista PDET no coinciden con los del MGN	Cruzar por código DIVIPOLA
El MultiPolygon de un municipio supera 16 MB	Medir el tamaño al cargar; simplificar con tolerancia fija solo si hace falta y documentarlo
Las dos fuentes dan conteos distintos para un municipio	Reportar ambas cifras y su diferencia; no promediarlas

8. Repositorio y responsables

Integrante	Aporte a la Entrega 1	Archivos
Miguel Duarte	Estructura del repositorio, configuración, diagrama E-R y primera versión del documento	`README.md`, `config.yaml`, `docker-compose.yml`, `diagrama_er.png`
Derek Sarmiento	Esquema implementado (validadores e índices) y este documento en LaTeX	`scripts/00_init_db.py`, `docs/delivery1/entrega1.tex`
José Medina	Verificación de la lista PDET (170 municipios, 16 subregiones)	`scripts/00_check_pdet_list.py`
Sebastián Sánchez Vásquez	Verificación del entorno y credenciales fuera del código	`scripts/00_check_environment.py`, `.env.example`
Samuel Emperador	Diagrama E-R editable en Mermaid	`docs/delivery1/diagrama_er.md`

Los datos crudos van en `data/` (ignorado por git) y los resultados en `outputs/`. El `README.md` indica el orden de ejecución.